

# PrimeGold

Self-hosted accounting, GST and inventory system for a trading business — one server on the owner's PC, used from a browser, an Android phone and a Windows app, with the books mirrored into Excel.

Project Report · Gopi Patel · Requirements, direction, AI-assisted development and delivery · August–September 2026

|                                                                     |                                                        |                                                                    |                                                                    |
|---------------------------------------------------------------------|--------------------------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------|
| <div>473</div> <div>unit tests passing<br/>re-run 16 Sep 2026</div> | <div>65</div> <div>data models<br/>13 migrations</div> | <div>239</div> <div>API route handlers<br/>17 server modules</div> | <div>3</div> <div>delivery targets from<br/>one web codebase</div> |
|---------------------------------------------------------------------|--------------------------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------|

Prime Gold

MASTERS

TRANSACTIONS

UTILITIES

REPORTS

COMPANY

Quit

← Back

Books from 2026-04-01

Company1

Administrator

Accountant

Sign out

Company

Create Company

Data

Exchange

Import

Export

Share

Print

People

Document

Cancel voucher

Delete

Back

← Sales

No. 13

TOTAL

₹32,162.46

06/Sep/2026

# Add reference

PARTICULARS

By

gogo

Debit

32,162.46

x

Cur Bal:

9,41,105.42 Dr

To

salesPlanks

Credit

27,256.32

x

Cur Bal:

89,330.64 Cr

To

CGST

Credit

2,453.07

x

Cur Bal:

4,906.14 Cr

To

SGST

Credit

2,453.07

x

Cur Bal:

4,906.14 Cr

+ Add a line

Debit

32,162.46

Credit

32,162.46

STOCK MOVEMENT

27,256.32

PARTICULARS

GODOWN

QUANTITY

RATE

PER

AMOUNT

60x74

Alt Godown

13

Nos

2096.64

Nos

27,256.32

x

18 MM

F2 Date

F3 Company

F4 Contra

F5 Payment

F6 Receipt

F7 Journal

F8 Sales

F9 Purchase

F10 Other Vouchers

I More Details

O Related Reports

L Optional

T Post-Dated

J Stat Adjustment

F12 Configure

Voucher entry on the sample company: a GST sales invoice with ledger balances shown as you type, a live debit/credit check, and stock lines carried on the same voucher.

## 02

## Project Overview

PrimeGold is a working accounting system built for one client: a plywood and door trading business that keeps its own books. It runs as a small server on the business's Windows PC. Staff use it in a browser on that machine, as an installed app on Android phones, and as a Windows app on a PC in another location — all against the same books, over a private network, with nothing published to the internet.

It covers the daily cycle end to end: masters, voucher entry, statements, GST returns, and stock with bundle tracking. Alongside it, a separate process mirrors every change into Excel workbooks, because the client's accountant works from spreadsheets.

|                    |                                                                                                                                                                        |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Client need</b> | Keep books, file GST and track stock without handing data to a hosted service, while reaching it from phones and a distant PC                                          |
| <b>Users</b>       | Owner and bookkeeping staff — keyboard-driven data entry, Indian accounting conventions                                                                                |
| <b>Runs on</b>     | One Windows PC (SQLite, no database server to administer); devices connect over a private Tailscale network                                                            |
| <b>Delivered</b>   | Web app / PWA, Android APK, Windows portable exe, Excel mirror, backup and packaging scripts                                                                           |
| <b>My role</b>     | Requirements, constraints and product decisions; implementation directed through Claude Code; reproduction of defects; testing on real devices; packaging and delivery |
| <b>Status</b>      | In use on the client's server PC with a live company. Builds delivered 14 Sep 2026; work committed 15 Sep 2026; refinements continue from client feedback              |

## 03

## Project Objectives

| # | Objective                                              | How it is judged                                                                                    |
|---|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| 1 | Books that cannot be made inconsistent by ordinary use | Unbalanced vouchers are refused; statements agree with each other; unit tests assert the invariants |
| 2 | Data entry as fast as a keyboard-only workflow demands | Every screen has a shortcut; Enter and Esc walk the fields; voucher type on function keys           |
| 3 | Indian statutory output that can actually be filed     | GSTR-1 and GSTR-3B computed from posted vouchers with statutory set-off, exported as JSON           |
| 4 | Stock answers the question the business asks           | Stock Summary reports bundles per bundle size and the loose remainder                               |
| 5 | Same books from the shop, a phone, or another city     | One private address; Android and Windows builds carrying it; no public exposure                     |
| 6 | Excel copies that stay level with the books            | Workbooks rewritten within seconds of a create, alter or delete, including removing emptied months  |
| 7 | Operable by people who do not administer systems       | One launcher starts everything; no autostart; plain-language status and error messages              |

## 04

## Problem & Requirements

Captured from the client during development — several arrived as annotated screenshots of the running app.

| #   | Requirement                                                                                 | Status       | Verification                                                 |
|-----|---------------------------------------------------------------------------------------------|--------------|--------------------------------------------------------------|
| R1  | Double-entry vouchers of every standard class that cannot be saved unbalanced               | Complete     | Posting unit tests; Balance Sheet tie-out                    |
| R2  | Day Book, Ledger, Trial Balance, P&L, Balance Sheet, Outstanding with ageing, Stock Summary | Complete     | Invariant tests; live check scripts                          |
| R3  | GST returns produced from the books, not re-keyed                                           | Complete     | GSTR-1 / GSTR-3B check scripts                               |
| R4  | Alter and delete a voucher from the Day Book; anything altered shows everywhere at once     | Complete     | check-party 5/5; browser test                                |
| R5  | Bundle counts per bundle size, recorded per purchase                                        | Complete     | check-bundles 7/7                                            |
| R6  | Excel files reflecting creates, alters and deletes in near real time                        | Complete     | check-delete-sync 8/8                                        |
| R7  | Keyboard navigation: Enter forward, Esc back, arrows in lists                               | Complete     | Manual verification on entry and master forms                |
| R8  | One server address for every device, usable away from the shop                              | Complete     | APK and exe contents inspected; remote start                 |
| R9  | One click starts everything; nothing starts with Windows                                    | Complete     | Supervisor script; autostart entry removed                   |
| R10 | Multi-user with roles, lockout and an audit trail                                           | Complete     | Auth code; users and roles check script                      |
| R11 | e-Invoice / e-Way Bill submission to government portals                                     | Payload only | Payloads built, validated and recorded; nothing is submitted |

## 05

## Solution Overview

| Part                   | Responsibility                                                                                                                                                                                                                                                            |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>packages/shared</b> | Money and quantity arithmetic, the reserved chart of accounts (28 groups, 22 voucher types), GSTIN checksum and tax split, roles and capabilities, payroll, TDS and interest calculators. Imported by both server and client, so one rule cannot drift into two.          |
| <b>packages/server</b> | Fastify API over Prisma and SQLite. A framework-free posting rulebook decides what may be written; 17 modules expose masters, vouchers, reports, GST, inventory, payroll, banking, users, data import/export. It also serves the built web client, so one port is enough. |
| <b>packages/web</b>    | React PWA: keyboard-first voucher entry with live validation, report screens with drill-down, print layouts, offline reading and a queued-write fallback. The same build is wrapped as the Android and Windows apps.                                                      |
| <b>scripts</b>         | A supervisor that starts everything with one click, the Excel mirror (Python), backups, Android build, Windows packaging, and 27 live check scripts.                                                                                                                      |

| 31 VOUCHERS                                                            |                                                   |          |         |              | 12,83,641.96  |
|------------------------------------------------------------------------|---------------------------------------------------|----------|---------|--------------|---------------|
| DATE                                                                   | PARTICULARS                                       | VCH TYPE | VCH NO. | DEBIT AMOUNT | CREDIT AMOUNT |
| 20-Aug-2026                                                            | ▸ <a href="#">Part payment received from gogo</a> | Receipt  | 2       |              | 40,000.00     |
| 25-Aug-2026                                                            | ▸ <a href="#">Paid jay against timber bill</a>    | Payment  | 6       | 50,000.00    |               |
|                                                                        | jay                                               |          |         | 50,000.00    |               |
|                                                                        | bom                                               |          |         |              | (-50,000.00)  |
| <a href="#">Alter this voucher</a> <a href="#">Delete this voucher</a> |                                                   |          |         |              |               |

Day Book with a voucher opened: alter or delete it in place. Deleting asks for confirmation; the edit log keeps what the voucher said.

06

## Key Features & Capabilities

| Area            | What works today                                                                                                                                                                                                   | In client use     |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| Accounting core | All standard voucher classes; create, alter, cancel, delete; bill-wise references; optional, post-dated and reversing vouchers; per-type numbering; period locks; audit trail                                      | Yes               |
| Reports         | Day Book, Ledger with running balance, Trial Balance, P&L, Balance Sheet, cash and bank books, registers, Outstanding with ageing, cash flow, ratios, exception reports, drill-down to source                      | Yes               |
| GST             | Rate hierarchy, HSN/SAC, eleven treatments, tax invoice and bill of supply, GSTR-1, GSTR-3B with statutory set-off, JSON export; uncertain invoices held out of the return with the reason                         | Yes               |
| Inventory       | Items, groups, godowns, alternate units, batches and expiry; FIFO, LIFO, weighted average, standard and last-purchase valuation; bundle breakdown per purchase; orders, delivery and receipt notes, stock journals | Yes               |
| People & access | Six roles plus capability rights; create, disable, unlock, reset password, change email; lockout on repeated failures                                                                                              | Yes               |
| Excel mirror    | Folder per company; workbooks per voucher type and month, plus Day Book, Trial Balance, Stock Summary, GST and masters                                                                                             | Yes               |
| Payroll         | Employees, pay heads, dated salary structures, attendance, PF/ESI/PT, payslips, Form 16 and 24Q                                                                                                                    | No — switched off |
| Extension       | API tokens, signed webhooks, custom fields, saved report views, multi-company consolidation                                                                                                                        | No                |

**Primary**

All stock groups

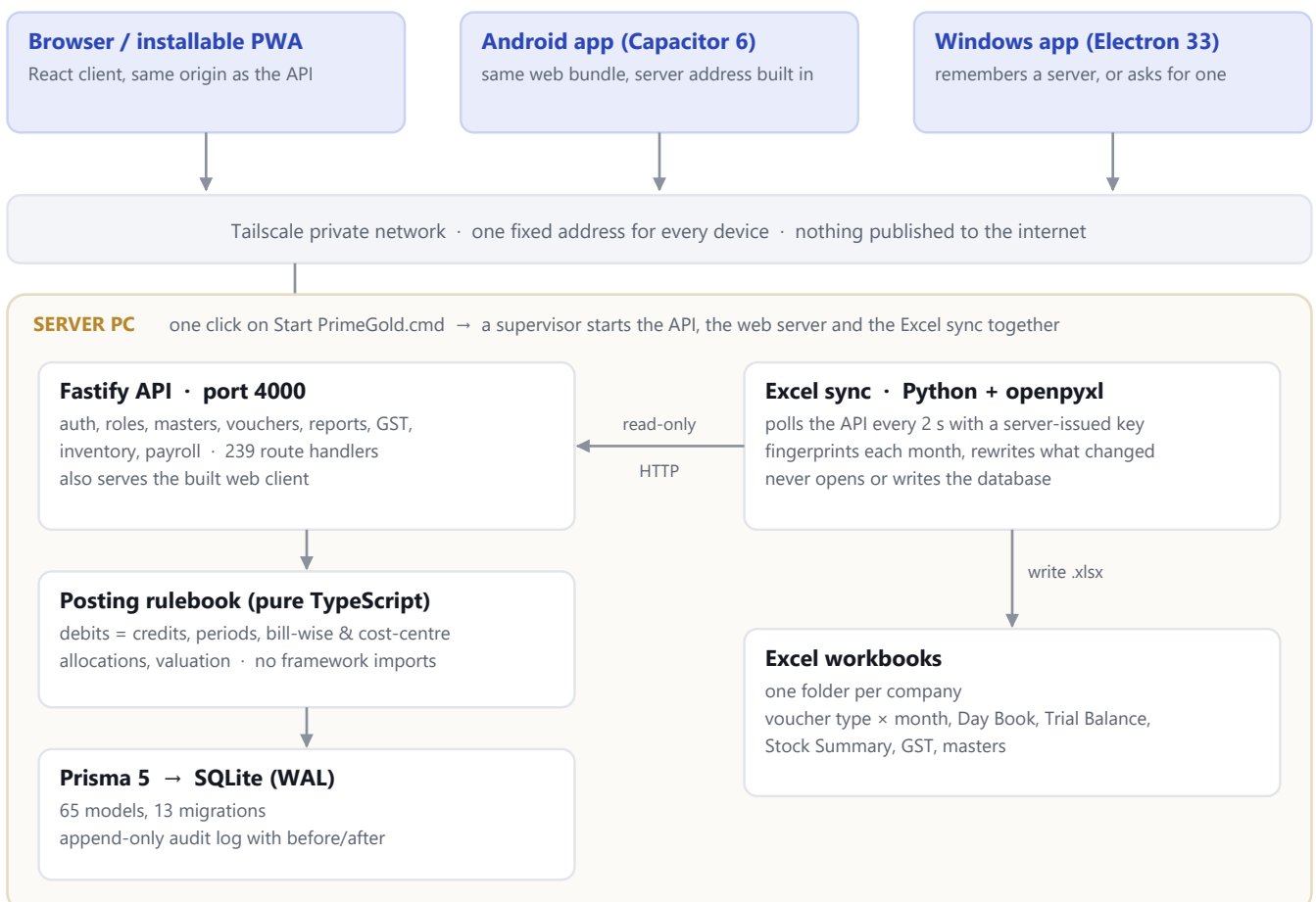
5 items in this group

| PARTICULARS                      | QUANTITY                                                          | RATE      | VALUE       |
|----------------------------------|-------------------------------------------------------------------|-----------|-------------|
| 60x74                            | 6.459 Nos                                                         | 2,096.65  | 13,543.68   |
| 78x65                            | 10 Nos                                                            | 35,208.00 | 3,52,080.00 |
| Plank <span>went negative</span> | -20 Kgs                                                           | —         | 0.00        |
| face7*4                          | 252 Nos<br>bundle of 25 = 10<br>loose = 2                         | 10.32     | 2,600.00    |
| fali(21x9)                       | 1050 Nos<br>bundle of 25 = 20<br>bundle of 30 = 10<br>loose = 250 | 25.61     | 26,891.50   |
| GRAND TOTAL                      |                                                                   |           | 3,95,115.18 |

Stock Summary: bundles still on hand by size and the loose remainder, under each item's quantity. Items that went negative are flagged, because that understates cost of sales.

07

# System Architecture



Three clients, one API, one database file. The Excel mirror is deliberately outside the server and reads the same API the app uses, so a fault in it can produce a wrong spreadsheet but never a wrong voucher.

## Layering inside the server

| Layer    | Rule it follows                                                                                                                                                              |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| domain/  | Pure functions over plain data — posting rules, valuation, the group tree. No Fastify, no Prisma. Unit-tested directly.                                                      |
| modules/ | HTTP routes and persistence. Every write path calls the rulebook; no route decides accounting policy on its own.                                                             |
| plugins/ | Session authentication and company context. Sessions are checked against the database on every request, so revoking access takes effect at once rather than at token expiry. |
| lib/     | Crypto, structured errors, the audit log, BigInt serialisation, mail, webhooks — shared plumbing with one owner each.                                                        |

## Data model

|                            |                                 |                      |                         |
|----------------------------|---------------------------------|----------------------|-------------------------|
| <b>65</b><br>Prisma models | <b>45</b><br>unique constraints | <b>98</b><br>indexes | <b>13</b><br>migrations |
|----------------------------|---------------------------------|----------------------|-------------------------|

Money columns are BIGINT holding paise. Enum-like columns are plain strings validated by Zod, and there are no JSON columns, so the schema moves to PostgreSQL by changing the datasource rather than the model.

08

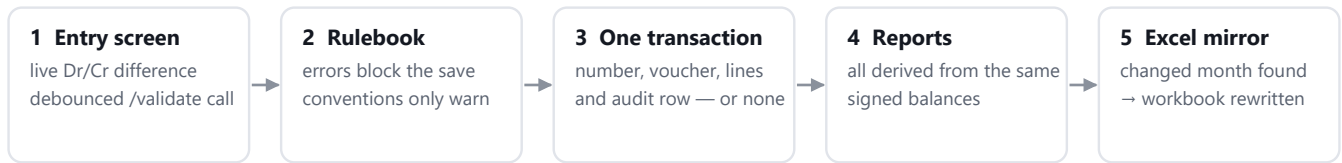
## Technology Stack

| Layer       | Technology                                         | Why                                                                              |
|-------------|----------------------------------------------------|----------------------------------------------------------------------------------|
| Language    | TypeScript across server, client and shared domain | One type system from database row to screen; shared rules imported by both sides |
| API         | Fastify 5, Zod, helmet, CORS, rate-limit           | Schema-validated routes; security headers and throttling as plugins              |
| Data        | Prisma 5, SQLite in WAL mode                       | No database server to administer on a shop PC; schema kept portable              |
| Client      | React 18, Vite 7, service worker, IndexedDB        | Installable PWA with offline reading and queued writes                           |
| Mobile      | Capacitor 6                                        | Wraps the exact web build — no second codebase                                   |
| Desktop     | Electron 33, electron-builder (portable)           | One .exe, no installer, no Node.js needed on the target PC                       |
| Integration | Python 3 with openpyxl                             | Mature Excel writing; runs beside the server and speaks only HTTP                |
| Network     | Tailscale                                          | One fixed private address that works on mobile data without opening router ports |
| Testing     | Vitest; Node scripts against the live API          | Fast pure-function tests plus end-to-end reconciliation over real HTTP           |

09

## Core Workflows

### Posting a voucher



The rulebook runs twice: debounced while typing, where a failed check never blocks the keyboard, and again on save, where an error rejects the whole write.

### Three more paths worth naming

| Workflow            | What happens                                                                                                                                                                                                                                                                                                          |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Creating a company  | One transaction seeds the reserved chart — 28 groups, 22 voucher types, GST tax ledgers, Cash and the P&L account. Either a company exists complete, or it does not exist.                                                                                                                                            |
| Print and come back | Save & Print writes the voucher, replaces the history entry with the saved voucher's own address, then opens the document. Back therefore returns to a filled, editable voucher rather than a blank form.                                                                                                             |
| Excel mirroring     | The watcher asks the API what exists, fingerprints each month with SHA-256, and rewrites only months whose fingerprint changed. A month whose vouchers have all gone has its workbook deleted, which is what makes a delete visible in Excel. A status file in each folder says, in plain language, when it last ran. |

10

## Important Engineering Decisions

Each is visible in the code today. Problem → Decision → Implementation → Result.

| Money is never a floating-point number |                                                                                                                                                                                                                                                                                                                                                                                    |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PROBLEM</b>                         | Floating-point sums drift, and one paisa of drift breaks a trial balance and a GST return.                                                                                                                                                                                                                                                                                         |
| <b>DECISION</b>                        | Carry money as integer paise, quantities as integer micro-units, rates as basis points.                                                                                                                                                                                                                                                                                            |
| <b>IMPLEMENTATION</b>                  | <code>shared/src/money.ts</code> is the only place arithmetic conventions live; values are bounds-checked; rounding is half away from zero because <code>Math.round(-2.5)</code> would bias credit notes. Stored as BIGINT, converted at one boundary in <code>lib/serialize.ts</code> , which refuses values beyond the safe integer range rather than rounding someone's ledger. |
| <b>RESULT</b>                          | The double-entry invariant reduces to <code>sum === 0</code> , and values round-trip through SQLite and JSON without loss.                                                                                                                                                                                                                                                         |

### A posting rulebook with no framework in it

|                       |                                                                                                                                                                                                                                           |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PROBLEM</b>        | Rules about what may be written risked spreading across routes, screens and import paths.                                                                                                                                                 |
| <b>DECISION</b>       | Keep every posting rule in pure functions over plain data.                                                                                                                                                                                |
| <b>IMPLEMENTATION</b> | server/src/domain/posting.ts holds balance, bill-wise and cost-centre allocation, period and per-class rules. Errors block; conventions return warnings. The same functions serve typed entry, the live check during typing, and imports. |
| <b>RESULT</b>         | Rules are unit-tested directly and cannot be bypassed by a new entry path.                                                                                                                                                                |

### Voucher numbers allocated inside the save

|                       |                                                                                                                                                                                        |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PROBLEM</b>        | Two clerks saving at the same moment could both be handed the same number by a read-then-write.                                                                                        |
| <b>DECISION</b>       | Allocate the number in the same transaction that writes the voucher.                                                                                                                   |
| <b>IMPLEMENTATION</b> | A counter row per voucher type, series and financial year is updated inside the caller's transaction; manual numbers rely on a composite unique index rather than a check-then-insert. |
| <b>RESULT</b>         | Concurrent saves serialise on the counter row; a failed save leaves no gap and no duplicate.                                                                                           |

### An audit log that can only grow

|                       |                                                                                                                         |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>PROBLEM</b>        | Statutory audit-trail expectations require every change to be reconstructible, and an editable log proves nothing.      |
| <b>DECISION</b>       | Write-only edit log with before and after snapshots for every create, alter, delete, cancel and sign-in event.          |
| <b>IMPLEMENTATION</b> | lib/audit.ts is the only writer; nothing in the codebase updates or deletes an audit row.                               |
| <b>RESULT</b>         | A deleted voucher leaves the books but not the history — which is what the Day Book tells the user before they confirm. |

### Excel as a mirror, outside the server

|                       |                                                                                                                                                                    |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PROBLEM</b>        | The client wanted Excel files that follow every change, but a spreadsheet must never be able to stop a voucher from saving.                                        |
| <b>DECISION</b>       | Build the mirror as a separate process that reads the public API and never the database.                                                                           |
| <b>IMPLEMENTATION</b> | A Python watcher polls every 2 s, fingerprints each month, rewrites only changed months, deletes workbooks for months with no data, and runs as a single instance. |
| <b>RESULT</b>         | Deletes reach Excel within seconds; if the mirror crashes or is never started, the books are untouched.                                                            |

### A server-issued key for the mirror, not a stored password

|                       |                                                                                                                                                                                   |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PROBLEM</b>        | The mirror signed in with the admin password from a config file. When the owner changed the password the mirror stopped, and its retries counted towards account lockout.         |
| <b>DECISION</b>       | Have the server mint a dedicated session for the mirror on every start.                                                                                                           |
| <b>IMPLEMENTATION</b> | lib/syncToken.ts revokes the previous key, creates a session labelled for the sync, and writes it to the data folder; the mirror re-reads it whenever the API answers 401 or 403. |
| <b>RESULT</b>         | Password changes no longer affect the mirror, restarts are picked up automatically, and no password is stored for automation.                                                     |

## One web codebase, three platforms

|                       |                                                                                                                                                                                                           |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PROBLEM</b>        | The client needed a phone app and a Windows app; three user interfaces would multiply every fix by three.                                                                                                 |
| <b>DECISION</b>       | Ship the same React build everywhere and make only the server address platform-aware.                                                                                                                     |
| <b>IMPLEMENTATION</b> | Vite injects a default address from one text file; Capacitor wraps the build for Android; Electron loads it and remembers an address, falls back to a local server, and otherwise shows a connect screen. |
| <b>RESULT</b>         | A fix lands once and reaches all three targets on the next build.                                                                                                                                         |

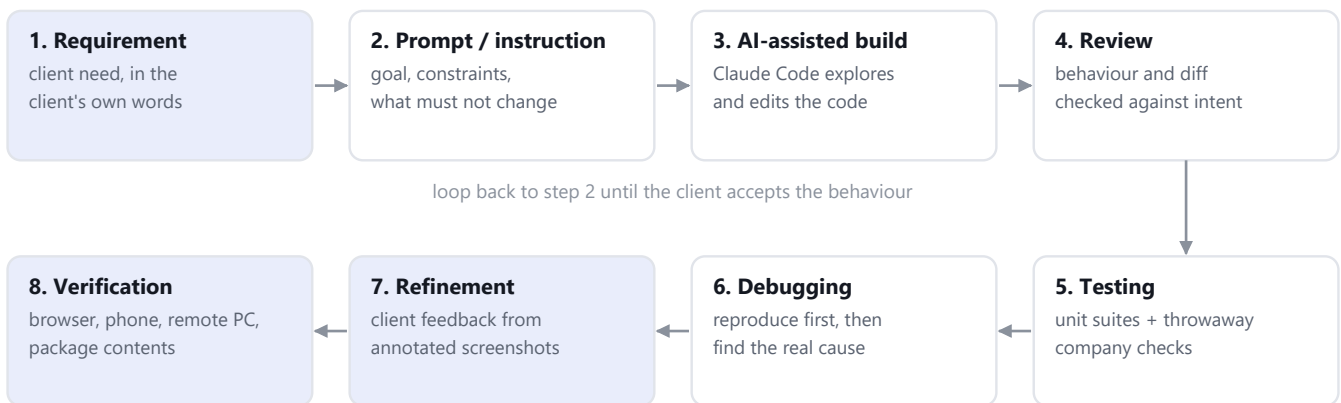
## Keyboard navigation that does not fight the screen

|                       |                                                                                                                                                                                                       |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PROBLEM</b>        | Enter submits web forms and Esc already closes lists and dialogs, so a naive global handler broke both.                                                                                               |
| <b>DECISION</b>       | Enter moves forward last; Esc moves back first; both stand down while a list or dialog is open.                                                                                                       |
| <b>IMPLEMENTATION</b> | <code>lib/keyboardNav.ts</code> listens for Enter in the bubble phase, skipping events a component already handled, and for Esc in the capture phase so it can decide before a screen navigates away. |
| <b>RESULT</b>         | Entry never leaves the keyboard, and pickers, dialogs and sign-in behave exactly as before.                                                                                                           |

11

# AI-Assisted Development

Claude Code was the implementation tool. Direction, judgement and acceptance stayed with me.



### I was responsible for

Capturing requirements from the business and turning them into precise, testable instructions  
 Constraints: a change freeze on the running server during later iterations, and never touching the client's company — tests run on a sample or throwaway company  
 Decisions where there was a real trade-off: bundles recorded per purchase, one address for every device, manual start instead of autostart  
 Reproducing faults on the live setup and rejecting fixes that did not hold in practice  
 Acceptance testing on phones, a remote PC, Excel and printed documents; deciding when to release

### Claude Code assisted with

Exploring the codebase and proposing a diagnosis  
 Writing and editing application code, migrations and styles  
 Writing unit tests and throwaway-company check scripts  
 Running test suites, builds, and Android and Windows packaging  
 Inspecting build outputs and writing documentation

## One iteration, end to end

| Step         | What happened                                                                                                                                                                                                              |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Requirement  | Client report: after Save & Print, pressing Back cleared the voucher; and a ledger renamed with the alter feature still printed its old name on the invoice and in the Day Book.                                           |
| Instruction  | Fix both; do not alter the running server; do not touch the client's company — use the sample company.                                                                                                                     |
| Diagnosis    | Back returned to the blank new-voucher route. The name was not stale: names are resolved at read time, but in voucher mode the party field is hidden, so altering a line left the stored party pointing at the old ledger. |
| Reproduction | A check script created a throwaway company, reproduced the wrong party, applied the corrected payload, asserted the Day Book, the voucher and a rename, then deleted the company.                                          |
| Fix          | Replace the history entry with the saved voucher before opening the print view; derive the party from the lines whenever the party field is not on screen.                                                                 |
| Verification | check-party 5/5; 473 unit tests; in the browser, Print → Back reopened the saved voucher with its lines intact; the test voucher was deleted afterwards.                                                                   |

**Guardrails.** Failed sign-ins lock the account after five attempts, so automated work authenticates with the server-issued key rather than a password. Builds are checked by opening the APK and the exe and confirming the current bundle and address are inside — never inferred from a successful exit code.

12

## Implementation Details

| Concern           | How it is handled                                                                                                                                                                                                                                              |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BigInt boundary   | Prisma returns BigInt for money columns and JSON cannot carry one. Conversion happens in a single place, bounds-checked, rather than by patching the BigInt prototype — which would hide overflow instead of reporting it.                                     |
| Error taxonomy    | Every deliberate failure is an <code>AppError</code> with a status, a machine code and a message written for the person reading it, so the error handler never guesses a status and internals never leak into a response.                                      |
| Group tree        | A company has at most a few hundred groups, so the tree is loaded once per request and walked in memory — faster than recursive SQL and no materialised-path column to maintain on every rename.                                                               |
| Sessions          | Opaque tokens, stored only as SHA-256 hashes, checked against the database on every request. That is one extra query versus a stateless token, and it makes revoking access immediate.                                                                         |
| Offline behaviour | The service worker caches the shell, caches reports stamped with when they were taken, and never caches anything that changes the books. A save that cannot reach the server is queued in IndexedDB and reported as queued — never as saved.                   |
| Printing          | Four layouts: A4, A5, 80mm thermal and dot matrix, the last replacing shading with ruled lines because a ribbon cannot print a background. The choice is remembered per browser.                                                                               |
| Bundle tracking   | A nullable bundle size on each stock line (one migration). The Stock Summary nets bundles in and out per size; the remainder is shown as loose, or as “taken from bundles” when more pieces went out than the loose stock could cover.                         |
| Excel workbooks   | One folder per company, a sub-folder per voucher type, one workbook per month, plus Day Book, Trial Balance, Stock Summary, GST and masters. Each folder carries a status file saying when the mirror last ran and what it would mean if that time were stale. |

```
export function assertSafeMoney(value: number, what = 'amount'): Paise {
  if (!Number.isFinite(value)) throw new MoneyError(`${what} is not a finite number`);
  if (!Number.isInteger(value)) throw new MoneyError(`${what} must be an integer count of paise`);
  if (!Number.isSafeInteger(value)) throw new MoneyError(`${what} exceeds the safe integer range`);
  return value;
}
```

Invoice No. **13**  
Ref. No.

Dated **6-Sep-26**

**Company1**  
raipur  
State Name : Chhattisgarh, Code : 22

**Tax Invoice**  
Party : **gogo**  
GSTIN/UIN : 25222222522234  
State Name : Chhattisgarh, Code : 22

| Description of Goods | HSN/SAC | Alt. Quantity     | Quantity         | Rate  | per | Disc. % | Amount             |
|----------------------|---------|-------------------|------------------|-------|-----|---------|--------------------|
| 60x74                |         | 400.83 Sqf        | 13.00 Nos        | 68.00 | Sqf |         | 27,256.32          |
|                      |         |                   |                  |       |     |         |                    |
| CGST                 |         |                   |                  |       |     |         | 2,453.07           |
| SGST                 |         |                   |                  |       |     |         | 2,453.07           |
| <b>Total</b>         |         | <b>400.83 Sqf</b> | <b>13.00 Nos</b> |       |     |         | <b>₹ 32,162.46</b> |

E & O E

Amount Chargeable (in words)  
**INR Thirty Two Thousand One Hundred Sixty Two and Forty Six paise Only**

Tax invoice generated from the saved voucher: HSN summary, CGST/SGST split and the amount in words. Sample company data.

## 13

# Challenges & Solutions

| Challenge                                         | Root cause                                                                                                                                        | Solution                                                                                                          | Verified by                    |
|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|--------------------------------|
| Deleting a voucher left it in Excel               | A month emptied by a delete had nothing new to write, so its old workbook survived; the mirror also depended on a password that had since changed | Per-month fingerprints; sweep workbooks for months with no data; server-issued key re-read on 401/403             | check-delete-sync 8/8          |
| A password change silently stopped the mirror     | The admin password was copied into a config file, and retries consumed sign-in attempts                                                           | Dedicated session minted per server start; previous key revoked; no stored password                               | Mirror resumed after restart   |
| Windows app unusable on a PC away from the server | It assumed the server was on the same machine and quit otherwise                                                                                  | Remote mode: remembered address → local server → connect screen; default address built in; menu item to change it | Package contents; remote start |
| Back from the print view cleared the entry        | History still held the blank new-voucher route                                                                                                    | Replace history with the saved voucher's route before printing                                                    | Browser test, sample company   |
| Renamed ledger appeared not to update             | The hidden party field kept the old ledger after a line was altered                                                                               | Derive the party from the lines when the field is not on screen                                                   | check-party 5/5                |

| Challenge                                              | Root cause                                                                                      | Solution                                                                                   | Verified by                              |
|--------------------------------------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|------------------------------------------|
| Bundles of different sizes within one item             | Stock was held only in pieces                                                                   | Nullable bundle size per stock line; summary nets bundles per size and shows the remainder | check-bundles 7/7                        |
| Enter and Esc clashing with lists and dialogs          | Global key handlers ran before components could claim the key                                   | Bubble-phase Enter, capture-phase Esc, both inactive over open layers                      | Manual, entry and master forms           |
| Rebuilding the client blanked the app for remote users | The server lists static files once at start-up, so a rebuilt bundle has a name it does not know | Restart after a client rebuild; recorded as a release step and as a limitation             | Asset served as JavaScript after restart |

14

## Testing & Verification

Three kinds of evidence, kept apart so none is mistaken for another.

### Automated — unit tests (Vitest)

| Package      | Files     | Tests      | Coverage in substance                                                                                                  |
|--------------|-----------|------------|------------------------------------------------------------------------------------------------------------------------|
| shared       | 4         | 62         | Money parsing and rounding, GST invoice rules, TDS thresholds, interest, finance helpers                               |
| server       | 17        | 308        | Posting rules, valuation methods, reports, ageing, GST and bank reconciliation, payroll, income tax, pricing, analysis |
| web          | 6         | 103        | Stock units and bundle arithmetic, entry helpers                                                                       |
| <b>Total</b> | <b>27</b> | <b>473</b> | <b>All passing, re-run 16 Sep 2026</b>                                                                                 |

Several are invariants rather than examples: the balance sheet ties to the profit & loss (assets less liabilities equals net profit), ageing buckets have no gaps or overlaps, GST and bank reconciliation conserve value, and funds flow ties to the change in working capital.

### Automated — live checks against a running server

27 Node scripts create a throwaway company through the HTTP API, post real vouchers, assert that the reports agree with each other and with the books, then delete the company. Results recorded during this delivery cycle:

| Script            | Checks | What it proves                                                                    |
|-------------------|--------|-----------------------------------------------------------------------------------|
| check-bundles     | 7 / 7  | Bundle sizes are stored per purchase and netted correctly in the Stock Summary    |
| check-delete-sync | 8 / 8  | A deleted voucher disappears from the API and from the Excel workbooks            |
| check-party       | 5 / 5  | The party follows the voucher's lines, and a rename shows in the Day Book at once |

**Not re-run for this report:** the full reconciliation suite (`npm run verify`). Its scripts sign in with the originally seeded password, which has since been changed; running them would fail and count towards the account lockout. Moving them to the server-issued key is listed under Future Improvements.

### Manual verification

- Browser checks on the sample company after each client-facing fix: Print → Back, Day Book delete, keyboard navigation, bundle display.
- Phones and a Windows PC reaching the server over the private network.

- Android APK opened and inspected: it carries the rebuilt bundle and the server address.
- Windows exe inspected: its archive holds the start-up logic, connect screen and default address; launched in remote mode.
- Excel folders watched while vouchers were created, altered and deleted.

15

## Security, Reliability & Data Integrity

| Area          | Measure                                                                                                                                                                                                                          | Where                               |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| Passwords     | script with a per-user salt, NFKC-normalised input and constant-time comparison                                                                                                                                                  | lib/crypto.ts                       |
| Sign-in       | Identical message for unknown user and wrong password, with a dummy hash check so timing matches; lockout after 5 failures for 15 minutes; login limited to 10 attempts per 5 minutes                                            | modules/auth, app.ts                |
| Sessions      | Opaque random tokens stored only as SHA-256 hashes; 12-hour expiry; checked per request so revocation is immediate                                                                                                               | plugins/auth.ts                     |
| Authorisation | Per-company roles (Owner, Manager, Accountant, DataEntry, Auditor, ViewOnly) with capability rights where a deny always wins                                                                                                     | shared/security.ts                  |
| Exposure      | Helmet headers, CORS allow-list, global rate limit; reachable only on the private network, with nothing published to the internet                                                                                                | app.ts                              |
| Integrations  | API tokens stored as hashes; webhooks signed with HMAC-SHA256; CSV export neutralises formula injection                                                                                                                          | modules/extend, lib/webhooks.ts     |
| Integrity     | Balanced-or-rejected vouchers; numbering inside the transaction; period locks; append-only audit log; imports applied whole or not at all; book dates stored as calendar dates so a voucher cannot slide a day across time zones | domain/posting.ts, modules/vouchers |
| Reliability   | SQLite in WAL mode; one supervisor starting all processes; the Excel mirror read-only and able to recover from key rotation; scheduled backups with retention; optional AES-256-GCM encryption at rest                           | scripts/                            |

16

## Deployment & Usage

| Target    | How it is delivered                                                                                                                                      | Notes                                                                                    |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Server PC | One launcher checks Node.js, runs first-time setup if needed, then a supervisor starts the API, the web server and the Excel mirror with labelled output | Started by hand at the client's request — nothing registers itself to start with Windows |
| Phones    | Android APK (5.6 MB) built from the same web bundle, server address built in and changeable on first run                                                 | Debug-signed; installed by allowing the source                                           |
| Other PCs | Portable Windows exe (71 MB), no installer, connects to the built-in address or asks for one                                                             | Unsigned; the private network client must be running                                     |
| Network   | One fixed private address, reachable on Wi-Fi or mobile data                                                                                             | No router ports opened; nothing on the public internet                                   |
| Excel     | A folder per company beside the project, refreshed by the mirror while the server runs                                                                   | Read-only mirror of the books, with a status file in each folder                         |

**Release checklist used for the September 2026 builds:** rebuild the web client, restart the server so it serves the new bundle, sync the build into the Android project, build the APK, package the exe, then open both artifacts and confirm the new bundle and the address are inside.

17

# My Role & Contribution

| Responsibility        | What I did                                                                                                                                                                                      |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Requirements          | Worked with the business to capture how it keeps books and stock, and turned requests — often annotated screenshots — into specific, testable instructions.                                     |
| Product decisions     | Chose between options where there was a real trade-off: bundle breakdown recorded per purchase; one address for every device; manual start rather than autostart; which rough edges could wait. |
| Engineering direction | Directed the implementation with Claude Code, set the constraints it worked under, and required every fix to be reproduced before it was accepted.                                              |
| Quality               | Tested on real devices and the live setup, reported failures back with evidence, and rejected changes that did not solve the problem in practice.                                               |
| Delivery              | Ran the server on the client's PC, set up the private network, and released the Android and Windows builds.                                                                                     |

**Attribution.** Most source code in this repository was generated by Claude Code under my direction. My contribution is the problem definition, the decisions, the constraints, the verification and the delivery — the parts that decide whether the software is right for the people using it.

18

# Results & Verified Metrics

|                                                                  |                                                                                 |                                                                        |                                                          |
|------------------------------------------------------------------|---------------------------------------------------------------------------------|------------------------------------------------------------------------|----------------------------------------------------------|
| <b>473</b><br>unit tests passing<br>27 files, re-run 16 Sep 2026 | <b>20 / 20</b><br>live checks passing this cycle<br>bundles, delete-sync, party | <b>57.8k</b><br>lines of TypeScript source<br>plus 5.5k lines of tests | <b>61</b><br>screens in the client                       |
| <b>239</b><br>API route handlers                                 | <b>65 / 45 / 98</b><br>models / unique constraints /<br>indexes                 | <b>5.6 / 71 MB</b><br>Android APK /<br>Windows exe                     | <b>2</b><br>commits, working tree clean<br>(15 Sep 2026) |

| Outcome                            | Evidence                                                                                                                             |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Books that balance by construction | The rulebook rejects unbalanced vouchers; the Balance Sheet reports its tie-out; invariant tests pass                                |
| A system in daily use              | Running on the client's PC with a live company; phones and a remote PC connected over the private network                            |
| Client-specific needs met          | Bundle breakdown (7/7), Day Book delete reaching Excel (8/8), party and rename behaviour (5/5), keyboard navigation, one-click start |
| Three platforms from one codebase  | Web, Android APK and Windows exe built from the same client bundle on 14 Sep 2026                                                    |
| Quality maintained while changing  | 473 unit tests passing after the latest fixes                                                                                        |

● The books tie. Debits and credits agree exactly.

| LIABILITIES         |             | ASSETS                |             |
|---------------------|-------------|-----------------------|-------------|
| ▸ Capital Account   | 52,000.00   | ▸ Current Liabilities | 96,793.86   |
| ▸ Loans (Liability) | 1,000.00    | ▸ Fixed Assets        | 50,000.00   |
| Profit & Loss A/c   | 4,05,323.71 | ▸ Current Assets      | 3,11,529.85 |
| Opening Balance     | 0.00        | Total                 | 4,58,323.71 |
| Current Period      | 4,05,323.71 |                       |             |
| Total               | 4,58,323.71 |                       |             |

Balance Sheet on the sample company. The tie-out banner is computed from the period's debits and credits, not decorative.

19

## Limitations

Stated plainly, including what is not finished.

- **Runs in development mode.** The server PC runs the API through a file-watching TypeScript runner and the client through the Vite dev server, rather than as a compiled production service.
- **A client rebuild needs a server restart.** Static files are listed once at start-up, so a rebuilt bundle is not served until PrimeGold restarts. Found during delivery and documented; not yet fixed in code, because the running server was under a change freeze.
- **The full live suite cannot run as it stands.** `npm run verify` signs in with the originally seeded password and must be moved to the server-issued key first.
- **Version history is coarse.** Two commits, the second bundling 48 files — useful as a snapshot, not as a reviewable history.
- **Unsigned builds.** The APK is debug-signed and the exe unsigned, so installation shows warnings, and neither updates itself.
- **Excel is one-way.** Edits made in the workbooks are not imported back, and the mirror polls every two seconds rather than receiving events.
- **Single machine.** The server PC must be switched on, every device needs the private-network client, and backups stay local unless copied elsewhere.
- **Statutory portals are not called.** e-Invoice and e-Way Bill payloads are built, validated and recorded, and GST returns export JSON, but nothing is submitted to a government portal.
- **Breadth exceeds field use.** Payroll, TDS, manufacturing and several advanced reports are implemented and unit-tested, but the client's company does not use them, so they are unproven in daily work.
- **Smaller items:** a single 706 kB JavaScript bundle (build warning); parts of the README predate recent changes; vouchers saved before the party fix keep their old party until re-saved; the sample company holds leftover test records.

20

## Future Improvements

---

- Run the server as a compiled Windows service, and read static files per request so a client rebuild needs no restart.
- Move the live check scripts to the server-issued key, then run them in CI against a disposable database alongside the unit tests.
- Commit in small reviewed changes with a changelog, so the history is reviewable.
- Push change events to the Excel mirror instead of polling, and add an import path for corrections made in the workbooks.
- Sign the Android and Windows builds and add in-app update checks.
- Browser end-to-end tests for keyboard-driven entry and printing.
- Encrypted off-machine backups, and a documented PostgreSQL move for multi-branch use.
- Split the client bundle by route to cut first-load time on phones.

21

## Conclusion

---

PrimeGold covers the whole path from a business's real workflow to software it runs every day: requirements taken from the people using it, an architecture that keeps accounting correct by design, and a delivery that reaches phones and distant PCs without putting the books on the internet.

The decisions that matter most are the unglamorous ones — integer money, a rulebook that cannot be bypassed, numbering inside the transaction, an append-only audit log, and a mirror that cannot damage the books it copies. AI-assisted development made this breadth reachable in weeks; the discipline around it — reproduce first, test on throwaway data, verify the real artifact — is what makes the result trustworthy.

What remains is written down rather than hidden: production hardening, a reviewable commit history, signed builds, and moving the live suite onto the server-issued key.

# Master Prompt — Rebuild This Project

The same specification written in each model's prompt structure. Any one of them is enough on its own.

## Claude

```
<role>
You are a senior full-stack engineer who builds accounting software for Indian businesses. You know
double-entry bookkeeping, GST (CGST/SGST/IGST, GSTR-1, GSTR-3B), inventory valuation, TypeScript,
Node.js, React and SQL.
</role>

<context>
A plywood and door trading business keeps its own books. Staff are bookkeepers who work from the
keyboard and expect Indian conventions: vouchers, Dr/Cr, ledgers under reserved groups, lakh/crore
digit grouping, a financial year from 1 April to 31 March. The books must stay on the business's own
PC, be reachable from phones and a PC in another location over a private network, and be mirrored
into Excel files for the accountant. Stock is bought in bundles of different sizes (for example 25 or
35 sheets) and sold by the piece. Nobody on site administers systems.
</context>

<instructions>
1. Build a monorepo with npm workspaces and TypeScript throughout:
   packages/shared, packages/server (Fastify 5 + Prisma 5 + SQLite WAL + Zod), packages/web
   (React 18 + Vite, installable PWA). Vitest for tests. Android via Capacitor 6. Windows via
   Electron packaged as a portable .exe.
2. packages/shared: money as integer paise, quantities as integer micro-units, rates as basis points,
   rounding half away from zero, safe-integer guards; the reserved chart (28 groups, 22 voucher
   types); GSTIN checksum and the CGST/SGST/IGST split; roles (Owner, Manager, Accountant,
   DataEntry, Auditor, ViewOnly) with capability rights where a deny wins; payroll, TDS and interest
   calculators.
3. server/src/domain: a posting rulebook as pure functions with no framework imports - signed amounts
   (+debit/-credit) that must sum to zero, bill-wise allocations (New Ref, Against Ref, Advance, On
   Account), cost-centre allocations, period locks, books-begin-from enforcement, per-voucher-class
   conventions where errors block and conventions only warn; valuation for FIFO, LIFO, weighted
   average, standard cost and last purchase; the group tree loaded per request and walked in memory.
4. server modules: auth (scrypt hashes; opaque session tokens stored as SHA-256 with 12-hour expiry;
   lockout after 5 failures for 15 minutes; identical message for unknown user and wrong password
   plus a dummy hash check so timing matches; login limited to 10 attempts per 5 minutes; helmet;
   CORS allow-list), users, company (creation seeds the whole chart in one transaction), masters,
   vouchers (create/alter/cancel/delete plus POST /vouchers/validate; number allocated inside the
   same transaction, per type, series and financial year; append-only audit log with before/after
   snapshots), reports (Day Book, Ledger with running balance, Trial Balance, P&L, Balance Sheet,
   Outstanding with ageing, cash flow, ratios), gst (GSTR-1, GSTR-3B with statutory ITC set-off, JSON
   export, invoices whose tax disagrees with their rate held back with the reason), inventory (items,
   units with alternate units, godowns, batches, stock lines carrying an optional bundleSize, a Stock
   Summary returning per-size bundle counts and the loose remainder), payroll, banking, data
   import/export. Serve the built web client from the API port. On start, mint a long-lived session
   for the Excel mirror, revoke the previous one, write it to data/sync.token.
5. packages/web: keyboard-first entry - Alt shortcuts per screen, F4-F9 for voucher types,
   Ctrl+Enter to save, Enter to the next field (document listener, bubble phase, skipping events a
   component already handled), Esc to the previous field (capture phase), both inactive while a list
   or dialog is open, arrow keys plus Enter in pickers. Voucher entry shows ledger balances in the
   autocomplete, a live difference, and calls /vouchers/validate debounced without ever blocking
   typing; when the party field is hidden the party is derived from the lines; Save & Print replaces
   the history entry with the saved voucher so Back reopens it. Day Book rows expand to "Alter this
   voucher" and "Delete this voucher" with confirmation. Print layouts for A4, A5, 80mm thermal and
   dot matrix. Offline: cache the shell, cache reports stamped with when they were taken, queue
   writes in IndexedDB and never report a queued save as saved.
6. Windows app: remembered address, then a locally running server, then a "Where are your books?"
   connect screen behind a preload bridge with no Node in the page; a menu item to change the
   address; never stop a server it did not start.
7. Excel mirror in Python with openpyxl: read only the API using data/sync.token, re-reading it on
   401/403; one folder per company; a workbook per voucher type per month plus Day Book, Trial
```

```

Balance, Stock Summary, GST and masters; --watch polls every 2 s, fingerprints each month with
SHA-256, rewrites only changed months, deletes workbooks for months with no data; single instance;
a plain-language status file.
8. Operations: one supervisor script that starts API, web and mirror with labelled output and keeps
the books running if Python is missing; a .cmd launcher that checks Node.js and runs first-time
setup; scripts for backup, Android build and Windows packaging. Nothing registers itself to start
with Windows.
9. Tests: unit tests for money, posting, valuation, GST set-off, ageing buckets with no gaps, and the
invariant "assets - liabilities = net profit"; live check scripts that create a throwaway company
through the API, post vouchers, assert the reports agree, then delete it.
10. Build in this order, running the tests after each step: shared -> posting rulebook -> auth and
company -> vouchers -> reports -> web entry and reports -> inventory and GST -> Excel mirror ->
packaging.
</instructions>

<constraints>
- Never use floating-point arithmetic for money.
- No Prisma enums or Json columns; validate enum-like strings with Zod so the schema moves to
PostgreSQL by changing only the datasource.
- Store book dates as calendar dates at UTC midnight.
- Never update or delete an audit row.
- The Excel mirror must never be able to block or alter a voucher save.
- Never test against real client books; use a sample or throwaway company.
- Scripts authenticate with the server-issued token, never a stored password, because failed
sign-ins lock the account.
- One web codebase for browser, Android and Windows.
</constraints>

<input>
No existing code. Start from an empty folder. The native apps read their default server address from
a single text file at build time (a private-network address on port 4000).
</input>

<examples>
<example>
Purchase of item "fali": 3 bundles of 25, 2 bundles of 35, 10 loose pieces; then a sale of 20 pieces.
Stock Summary row:
  fali   135 Nos
          bundle of 25 = 3
          bundle of 35 = 2
          taken from bundles = 10
</example>
<example>
A Payment voucher with lines Cash -1,000.00 and Rent +900.00.
The save is rejected before anything is written, the error names the 100.00 difference, and no
voucher number is consumed. Corrected to +1,000.00 it saves, takes the next number for its type,
series and year, and writes one audit row; the month's Payment and Day Book workbooks show it within
about two seconds, and deleting it removes it from both.
</example>
</examples>

<output_format>
1. A short plan listing packages and build order.
2. The code, built in the order above, with the test pass count reported after each step.
3. A README: first-time setup, the launcher, connecting phones and remote PCs, building the APK and
the exe.
4. A closing summary: what was built, test results with automated and manual kept separate, and
known limitations.
</output_format>

<final_check>
Verify before finishing: every voucher path goes through the rulebook and sums to zero; no float
touches money; Balance Sheet, P&L and Trial Balance agree on the sample company; a deleted voucher
disappears from the Excel files within seconds; Back from the print view reopens an editable voucher;
renaming a ledger changes the Day Book and the invoice at once; Enter and Esc never break lists or
dialogs; the APK and exe contain the current web bundle and the default address; all unit tests pass.
Report anything that does not pass rather than hiding it.
</final_check>

```

## OpenAI

**Role:** Senior full-stack engineer building a self-hosted accounting, GST and inventory system ("PrimeGold") for an Indian plywood and door trading business. You own implementation end to end: data model, API, web client, Android and Windows packaging, Excel integration and tests.

### Personality

Precise and plain-spoken. Explain trade-offs in one or two sentences, never in marketing language. Prefer boring, verifiable solutions over clever ones. Say when something is unverified.

### Goal

A working system the business runs on its own Windows PC: staff post vouchers from the keyboard in a browser, on an Android phone or in a Windows app on another PC, all against one server on a private network, with the books mirrored into Excel workbooks within seconds of every change.

### Success criteria

- A voucher cannot be written unless its signed amounts sum to zero; money is integer paise only.
- Balance Sheet, Profit & Loss and Trial Balance agree on the same data; unit tests assert "assets - liabilities = net profit" and that ageing buckets have no gaps.
- Voucher numbering is allocated inside the save transaction, per type, series and financial year: concurrent saves produce no duplicates and no gaps.
- Every create, alter, cancel and delete leaves an append-only audit row with before/after state.
- Stock Summary reports bundles per bundle size plus the loose remainder for items bought in bundles.
- GSTR-1 and GSTR-3B are computed from posted vouchers, with statutory ITC set-off and JSON export; an invoice whose tax disagrees with its rate is held out of the return with the reason shown.
- Deleting a voucher removes it from the Excel workbooks within seconds, including deleting a workbook whose month no longer has data.
- The same web build runs as browser app, Android APK and Windows portable exe.
- Unit tests and live API check scripts pass; the live scripts create and delete their own company.

### Constraints

- Stack: npm workspaces; TypeScript; Fastify 5, Prisma 5, SQLite in WAL mode, Zod on the server; React 18 + Vite PWA on the client; Capacitor 6 for Android; Electron for Windows; Python with openpyxl for the Excel mirror; Vitest for tests.
- Posting rules live in pure functions with no framework imports, so they can be unit-tested and cannot be bypassed by a new entry path.
- No Prisma enums or Json columns; enum-like values are Zod-validated strings, keeping a PostgreSQL move to a datasource change. Book dates are calendar dates stored at UTC midnight.
- Security: bcrypt password hashes; opaque session tokens stored as SHA-256 with 12-hour expiry; lockout after 5 failed sign-ins for 15 minutes; identical error for unknown user and wrong password; login limited to 10 attempts per 5 minutes; helmet and a CORS allow-list; six roles with capability rights where a deny wins.
- The Excel mirror reads only the API, authenticating with a token the server mints at start (data/sync.token) and re-reads on 401/403. It must never be able to block or alter a save.
- Nothing registers itself to start with Windows; one launcher starts the API, the web server and the mirror together.
- Never test against real client books. Never store a password for automation.

### Output

Working code in the repository, plus a README covering setup, the launcher, connecting phones and remote PCs, and building the APK and exe. Report test counts after each milestone. Keep explanations short; prefer tables and short lists over prose.

### Stop rules

- If a requirement is ambiguous, implement the conservative reading and state the assumption.
- If a check fails, fix the cause rather than the test, and report the failure.
- Stop and ask before anything that would touch real client data or expose the server publicly.
- Do not claim a feature works until its test or a live check passes.

## GPT-4.1

### # Role and Objective

You are a senior full-stack engineer. Build "PrimeGold", a self-hosted multi-user accounting, GST and inventory system for an Indian plywood and door trading business, runnable on one Windows PC and usable from a browser, an Android phone and a Windows app on another PC over a private network.

### # Instructions

- Create an npm-workspaces monorepo in TypeScript: shared, server, web.
- Keep all posting rules in pure functions; keep money exact; keep the audit trail append-only.
- Deliver the same web build as browser app, Android APK (Capacitor) and Windows portable exe (Electron).
- Mirror the books into Excel workbooks with a separate read-only process.

### ## Sub-categories

#### ### Shared domain

- Money: integer paise; quantities: integer micro-units; rates: basis points; round half away from zero; reject non-integers and unsafe ranges.
- Reserved chart: 28 groups, 22 voucher types. GSTIN checksum, CGST/SGST/IGST split.
- Roles: Owner, Manager, Accountant, DataEntry, Auditor, ViewOnly, plus capability rights (deny wins).

#### ### Server (Fastify 5, Prisma 5, SQLite WAL, Zod)

- Posting rulebook: signed amounts summing to zero; bill-wise and cost-centre allocations; period locks; books-begin-from; per-class conventions where errors block and conventions warn.
- Valuation: FIFO, LIFO, weighted average, standard, last purchase.
- Auth: scrypt hashes; SHA-256-stored opaque sessions, 12-hour expiry; lockout 5 failures / 15 minutes; identical failure message; login rate limit 10 per 5 minutes; helmet; CORS allow-list.
- Vouchers: create, alter, cancel, delete, and POST /vouchers/validate; number allocated inside the save transaction per type/series/financial year; append-only audit rows with before/after.
- Reports: Day Book, Ledger, Trial Balance, P&L, Balance Sheet, Outstanding with ageing, cash flow, ratios - all derived from the same signed balances.
- GST: GSTR-1, GSTR-3B with statutory ITC set-off, JSON export, uncertain invoices held back.
- Inventory: items, alternate units, godowns, batches; stock lines with optional bundleSize; Stock Summary returns per-size bundle counts and the loose remainder.
- Company creation seeds the full chart in one transaction. Serve the built web client from the API port. On start, mint and store a session token for the Excel mirror and revoke the previous one.

#### ### Web (React 18 + Vite PWA)

- Keyboard: Alt shortcuts, F4-F9 voucher types, Ctrl+Enter save, Enter = next field (bubble phase, skip handled events), Esc = previous field (capture phase), both inactive over open layers.
- Voucher entry: ledger autocomplete with balances, live difference, debounced validation that never blocks typing, party derived from lines when its field is hidden, Save & Print replacing history so Back reopens the saved voucher.
- Day Book alter and confirmed delete; print layouts A4, A5, 80mm thermal, dot matrix; offline shell cache, stamped report cache, IndexedDB write queue.

#### ### Delivery

- Android: Capacitor wrapping the same bundle. Windows: Electron portable exe with remembered address, local-server fallback and a connect screen. One supervisor script starts API, web and the Excel mirror; no autostart with Windows.

### # Reasoning Steps

1. Model the data and write the posting rulebook first; unit-test it before any HTTP route exists.
2. Add auth and company provisioning, then vouchers, then reports; keep reports derived, never stored.
3. Build the web client against the running API; add inventory and GST once posting is proven.
4. Add the Excel mirror last; it must be removable without affecting the books.

### # Output Format

Code in the repository; a README; after each milestone a short status line with the test pass count; at the end a summary separating automated tests from manual checks, and a list of known limitations.

### # Examples

#### ## Example 1

Input: Purchase 3 bundles of 25 and 2 bundles of 35 of item "fali", plus 10 loose; then sell 20.

Output: Stock Summary shows 135 Nos, "bundle of 25 = 3", "bundle of 35 = 2", "taken from bundles = 10".

### # Context

Users are bookkeepers working from the keyboard; the server runs on the shop's own PC; devices reach it over a private network on port 4000; the accountant wants Excel copies that follow deletions too.

### # Final instructions

Before finishing, verify: no float money anywhere; statements agree on sample data; deletes reach Excel within seconds; voucher numbers never collide; all tests pass. Report anything unverified.

## Codex

### Objective

Build "PrimeGold": a self-hosted accounting, GST and inventory system for an Indian plywood and door trading business, running on one Windows PC and used from a browser, an Android app and a Windows app over a private network, with every change mirrored into Excel workbooks.

### Context

Bookkeepers enter vouchers from the keyboard using Indian conventions (Dr/Cr, reserved groups, lakh/crore grouping, financial year 1 April to 31 March). The business keeps its data on its own machine and also wants Excel copies for its accountant. Stock is bought in bundles of different sizes and sold by the piece. No system administrator is available on site.

### Current State

Empty repository. No code, no database, no build scripts.

### Requirements

- \* npm-workspaces monorepo in TypeScript: packages/shared, packages/server, packages/web.
- \* Server: Fastify 5, Prisma 5, SQLite (WAL), Zod. Pure posting rulebook in server/src/domain.
- \* Money as integer paise, quantities as micro-units, rates as basis points; signed amounts where a voucher must sum to zero.
- \* Voucher create/alter/cancel/delete plus POST /vouchers/validate; numbering allocated inside the save transaction per voucher type, series and financial year; append-only audit log with before/after snapshots.
- \* Auth: script hashes, opaque sessions stored as SHA-256 with 12-hour expiry, lockout after 5 failed sign-ins for 15 minutes, identical failure message, login rate limit 10 per 5 minutes, helmet, CORS allow-list, six roles with capability rights.
- \* Reports: Day Book, Ledger, Trial Balance, P&L, Balance Sheet, Outstanding with ageing, cash flow, ratios. GST: GSTR-1 and GSTR-3B with statutory ITC set-off and JSON export.
- \* Inventory: items, alternate units, godowns, batches, five valuation methods, stock lines with an optional bundleSize, and a Stock Summary reporting bundles per size plus the loose remainder.
- \* Web client: React 18 + Vite PWA, keyboard-first voucher entry, Day Book alter and confirmed delete, print layouts (A4, A5, 80mm thermal, dot matrix), offline report cache and IndexedDB write queue.
- \* Android APK via Capacitor 6 and a Windows portable exe via Electron, both from the same web build.
- \* Excel mirror: Python + openpyxl, reading only the API with data/sync.token, --watch polling every 2 s, SHA-256 fingerprint per month, rewriting changed months and deleting empty ones.
- \* One supervisor script starting API, web and mirror together, plus a .cmd launcher.

### Constraints

- \* No floating-point money anywhere.
- \* No Prisma enums or Json columns; enum-like values are Zod-validated strings.
- \* Book dates stored as calendar dates at UTC midnight; audit rows never updated or deleted.
- \* The Excel mirror must never write to the database or be able to block a save.
- \* Files that matter: packages/shared/src/money.ts, packages/server/src/domain/posting.ts, packages/server/prisma/schema.prisma, packages/web/src/pages/VoucherEntry.tsx, scripts/export-excel.py, scripts/dev.mjs.
- \* Nothing may register itself to start with Windows.
- \* Automation authenticates with the server-issued token, never a stored password.

### Acceptance Criteria

- \* An unbalanced voucher is rejected and consumes no voucher number.
- \* Concurrent saves of the same voucher type never receive the same number.
- \* Balance Sheet, P&L and Trial Balance agree on seeded sample data.
- \* Renaming a ledger changes the Day Book and printed invoice immediately.
- \* Deleting a voucher removes it from the API and from the Excel workbooks within seconds.
- \* Stock Summary shows per-size bundle counts and the loose remainder.
- \* The APK and exe open the books using the built-in default address without being reconfigured.

### Verification

- \* Run the unit suites in all three packages and report pass counts.
- \* Run the live check scripts, which create a throwaway company, post vouchers, assert the reports agree and then delete the company.
- \* Verify a delete reaches Excel by watching the workbook folder while deleting a voucher.
- \* Confirm the packaged APK and exe contain the current web bundle and the default server address.

### Deliverable

A working repository with setup, launcher and packaging scripts; a README; passing unit and live checks; and a short report separating automated results from manual checks, with known limitations listed.

## Gemini

```

<role>
You are a senior full-stack engineer building accounting software for Indian businesses.
</role>

<instructions>
1. Build "PrimeGold" as an npm-workspaces TypeScript monorepo: shared domain, Fastify 5 + Prisma 5 +
  SQLite (WAL) server, React 18 + Vite PWA client.
2. Keep money as integer paise, quantities as micro-units and rates as basis points; round half away
  from zero; never use floating point for money.
3. Put every posting rule in pure functions: signed amounts summing to zero, bill-wise and
  cost-centre allocations, period locks, per-class conventions where errors block and conventions
  warn.
4. Allocate voucher numbers inside the save transaction, per voucher type, series and financial year,
  and write an append-only audit row with before/after state for every change.
5. Implement reports (Day Book, Ledger, Trial Balance, P&L, Balance Sheet, Outstanding with ageing,
  cash flow, ratios) derived from the same signed balances, and GST returns (GSTR-1, GSTR-3B with
  statutory ITC set-off, JSON export).
6. Implement inventory with alternate units, godowns, batches and five valuation methods; stock lines
  carry an optional bundle size, and the Stock Summary reports bundles per size and the loose
  remainder.
7. Implement authentication: scrypt hashes, opaque sessions stored as SHA-256 with a 12-hour expiry,
  lockout after 5 failures for 15 minutes, an identical message for unknown user and wrong password,
  a login rate limit, helmet and a CORS allow-list, six roles with capability rights.
8. Make the client keyboard-first: Alt shortcuts, F4-F9 voucher types, Enter to the next field in the
  bubble phase, Esc to the previous field in the capture phase, both inactive while a list or dialog
  is open.
9. Package the same web build as an Android APK (Capacitor) and a Windows portable exe (Electron),
  each with a default server address read from one text file at build time.
10. Add a Python + openpyxl mirror that reads only the API with a server-issued token, polls every
    2 seconds, fingerprints each month, rewrites changed months and deletes workbooks for months with
    no data.
11. Write unit tests for money, posting, valuation, GST set-off and ageing, plus live check scripts
    that create and delete their own company through the API.
</instructions>

<constraints>
- No Prisma enums or Json columns; validate enum-like strings with Zod so PostgreSQL needs only a
  datasource change.
- Book dates are calendar dates stored at UTC midnight; audit rows are never updated or deleted.
- The Excel mirror must never write to the database or block a voucher save.
- Never test against real client books; never store a password for automation.
- Nothing may register itself to start with Windows.
</constraints>

<output_format>
Code in the repository, a README covering setup and packaging, test pass counts after each milestone,
and a closing summary that separates automated tests from manual checks and lists known limitations.
</output_format>

<context>
Users are bookkeepers who work from the keyboard. The server runs on the business's own Windows PC
and is reached by phones and another PC over a private network on port 4000. The accountant works
from Excel copies of the books, which must follow deletions as well as additions. Stock is bought in
bundles of different sizes and sold by the piece.
</context>

<task>
Implement the system in this order, running the tests after each step: shared domain, posting
rulebook, auth and company provisioning, vouchers, reports, web voucher entry and reports, inventory
and GST, Excel mirror, packaging.
</task>

<final_instruction>
Before finishing, verify that statements agree on sample data, that an unbalanced voucher is rejected
without consuming a number, that a delete reaches the Excel workbooks within seconds, and that all
unit tests and live checks pass. State anything that could not be verified.
</final_instruction>

```

## Universal structure

### ROLE

Senior full-stack engineer building accounting software for Indian businesses.

### OBJECTIVE / GOAL

Build "PrimeGold": a self-hosted, multi-user accounting, GST and inventory system that runs on one Windows PC and is used from a browser, an Android phone and a Windows app over a private network, with the books mirrored into Excel workbooks within seconds of every change.

### CONTEXT

A plywood and door trading business keeps its own books. Bookkeepers work from the keyboard using Indian conventions (Dr/Cr, reserved groups, lakh/crore grouping, financial year 1 April to 31 March). Stock is bought in bundles of different sizes and sold by the piece. The accountant wants Excel copies. No system administrator is on site.

### REQUIREMENTS / INSTRUCTIONS

1. Monorepo: npm workspaces, TypeScript. shared (domain), server (Fastify 5, Prisma 5, SQLite WAL, Zod), web (React 18, Vite, PWA). Vitest for tests.
2. Money as integer paise, quantities as micro-units, rates as basis points; signed amounts where +debit and -credit must sum to zero before a voucher is written.
3. Pure posting rulebook: allocations, period locks, per-class conventions; errors block, conventions warn. Valuation: FIFO, LIFO, weighted average, standard, last purchase.
4. Vouchers: create, alter, cancel, delete, validate; numbering inside the save transaction per type, series and financial year; append-only audit log with before/after snapshots.
5. Reports derived from the same balances: Day Book, Ledger, Trial Balance, P&L, Balance Sheet, Outstanding with ageing, cash flow, ratios. GST: GSTR-1, GSTR-3B with statutory ITC set-off and JSON export, holding back invoices whose tax disagrees with their rate.
6. Inventory: items, alternate units, godowns, batches; stock lines with an optional bundle size; the Stock Summary reports bundles per size and the loose remainder.
7. Identity: script hashes; opaque sessions stored as SHA-256, 12-hour expiry; lockout after 5 failures for 15 minutes; identical failure message; login rate limit; six roles with capability rights where a deny wins.
8. Web client: keyboard-first entry (Enter forward in the bubble phase, Esc back in the capture phase, both inactive over open layers), live validation that never blocks typing, Day Book alter and confirmed delete, print layouts for A4, A5, thermal and dot matrix, offline report cache and an IndexedDB write queue.
9. Delivery: Android APK via Capacitor and a Windows portable exe via Electron from the same build, each with a default server address; one supervisor script starting API, web and the Excel mirror; no autostart with Windows.
10. Excel mirror: Python + openpyxl, API-only with a server-issued token, 2-second polling, SHA-256 fingerprint per month, changed months rewritten, empty months removed.

### CONSTRAINTS

- No floating-point money. No Prisma enums or Json columns. Book dates at UTC midnight.
- Audit rows are never updated or deleted.
- The Excel mirror cannot write to the database or block a save.
- Automation uses the server-issued token, never a stored password.
- Never test against real client books.

### SUCCESS CRITERIA

- Unbalanced vouchers are rejected and consume no number; concurrent saves never collide.
- Statements agree on sample data; ageing buckets have no gaps.
- A rename shows everywhere at once; a delete reaches Excel within seconds.
- Bundle counts are correct per size, with the remainder shown as loose or taken from bundles.
- All unit tests and live check scripts pass.

### EXAMPLES

Purchase 3 bundles of 25 and 2 bundles of 35 of "fali", plus 10 loose, then sell 20 pieces ->  
Stock Summary: 135 Nos, bundle of 25 = 3, bundle of 35 = 2, taken from bundles = 10.

### OUTPUT FORMAT

Repository code, a README (setup, launcher, remote access, packaging), test pass counts per milestone, and a final summary separating automated from manual verification with known limitations.

### VERIFICATION / STOP CONDITIONS

Run all unit suites and the live checks; watch a workbook folder while deleting a voucher; open the packaged APK and exe to confirm they carry the current bundle and address. If a check fails, fix the cause rather than the test. Do not claim a feature works before its check passes.